

Chapter 1

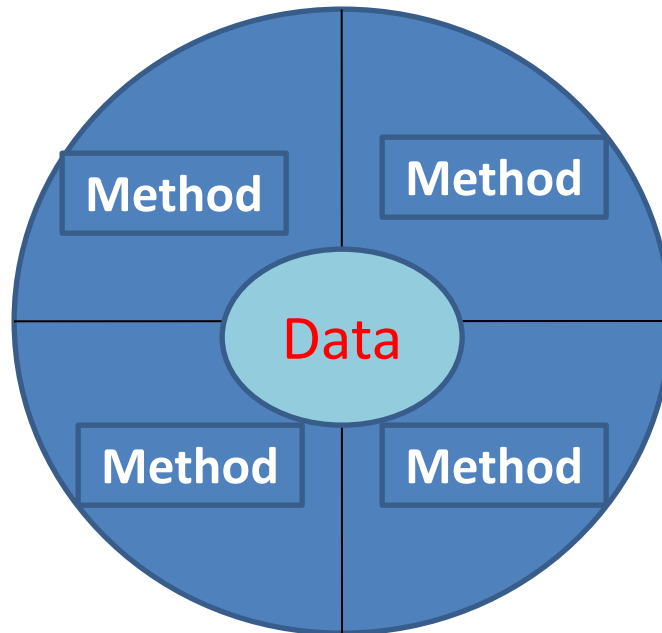
Introduction

Object Oriented Paradigm

- Major objectives: To eliminate flaws of procedural approach
- OOP treats data as critical element of the program.
- Hence does not allow it to flow freely.
- Ties data more closely to functions.
- Decomposes problem into number of entities called **objects**.

Object Oriented Paradigm

- Builds data and functions around these entities.
- Object = data + Methods



Features of Object Oriented Paradigm

- Emphasis is on data
- Programs are divided into: OBJECTS
- Data structures are designed such that they characterize objects.
- Methods operating on data of an object are tied together in the data structure.
- Data is hidden and cannot be accessed by external functions.
- Objects may communicate with each other through methods.
- Follows bottom up approach in program design.

Basic Concepts of Object Oriented Programming

- Objects and Classes
- Data Abstraction and Encapsulation
- Inheritance
- Polymorphism
- Compile time and run time mechanisms
- Dynamic binding
- Message Communication

Benefits of OOP

- Inheritance eliminates redundant code and extends the use of existing classes.
- Saves development time & increases productivity.
- Principles of data hiding helps programmers to build secure programs.
- Multiple objects coexist without any interference.

Benefits of OOP

- Objects in the problem domain can be mapped easily with objects in program domain.
- Work in project based on objects can easily be partitioned.
- OO systems can easily upgrade from small scale to large scale systems.
- Software complexity can easily be managed.
- Message passing techniques for communication between objects make the interface descriptions with external systems much simpler.

Applications of OOP

- Real time systems
- Simulation and modeling
- Object Oriented Databases
- Artificial intelligence and expert systems
- Neural networks
- Parallel programming
- Office automation systems
- Decision support systems
- CAD/CAM systems

INTRODUCTION TO JAVA

- Java is an Object Oriented Programming language.
- Java was developed by Sun Microsystems in 1991.
- Initially called as ***Oak***

Why Java is important to internet?

- Java has a profound effect on the internet.
- Reason: Java allows the objects to move freely in the cyberspace.
- Categories of objects transmitted between the server and your PC in a network:
 - Passive information
 - Dynamic, active programs

Why java is important to internet

- Passive information Ex. : reading email, downloading a program (code is still passive)
- Dynamic objects: self executing programs.
- Such programs is an active agent on the client computer, yet is initiated by the server.

Why java is important to internet

- Java can create 2 types of programs:
 - **Applications:** A program that runs on your computer under the operating system of that computer
 - **Applets:** An application designed to be transmitted over the internet and executed by Java compatible web browser.
 - It is a tiny java program, dynamically downloaded across the network.

Why java is important to internet

- **Security:** Java applets can be downloaded safely without fear of viral infections or malicious intent.
- It confines a Java Program to the Java execution environment.
- Does not allow it to access other parts of the computer.
- **Java Programs are portable.**

Java Byte Code

- The output of Java compiler is not executable code, it is **Bytecode**.
- **Bytecode**: Highly optimized set of instructions designed to be executed by Java run-time system, which is called **Java Virtual Machine (JVM)**.
- JVM is an interpreter for bytecode.

Java Byte Code



- First, a Java program (source code) is compiled by the Java compiler into **Bytecode**.
- Bytecodes are not machine instructions.
- Hence in the second stage, java interpreters generates machine code.
- It can now be executed by the machine that is running the Java program.
- **This is how portability is achieved.**

Portability

- Java programs can be easily moved from one computer to another.
- Changes and upgrades in OS, processors and system resources will not force any changes in Java program.
- JVM needs to be implemented on each platform.
- Once the runtime package exists for a given system, any java program can run on it.
- Details of JVM differs from platform to platform, all interpret the same Java bytecode.

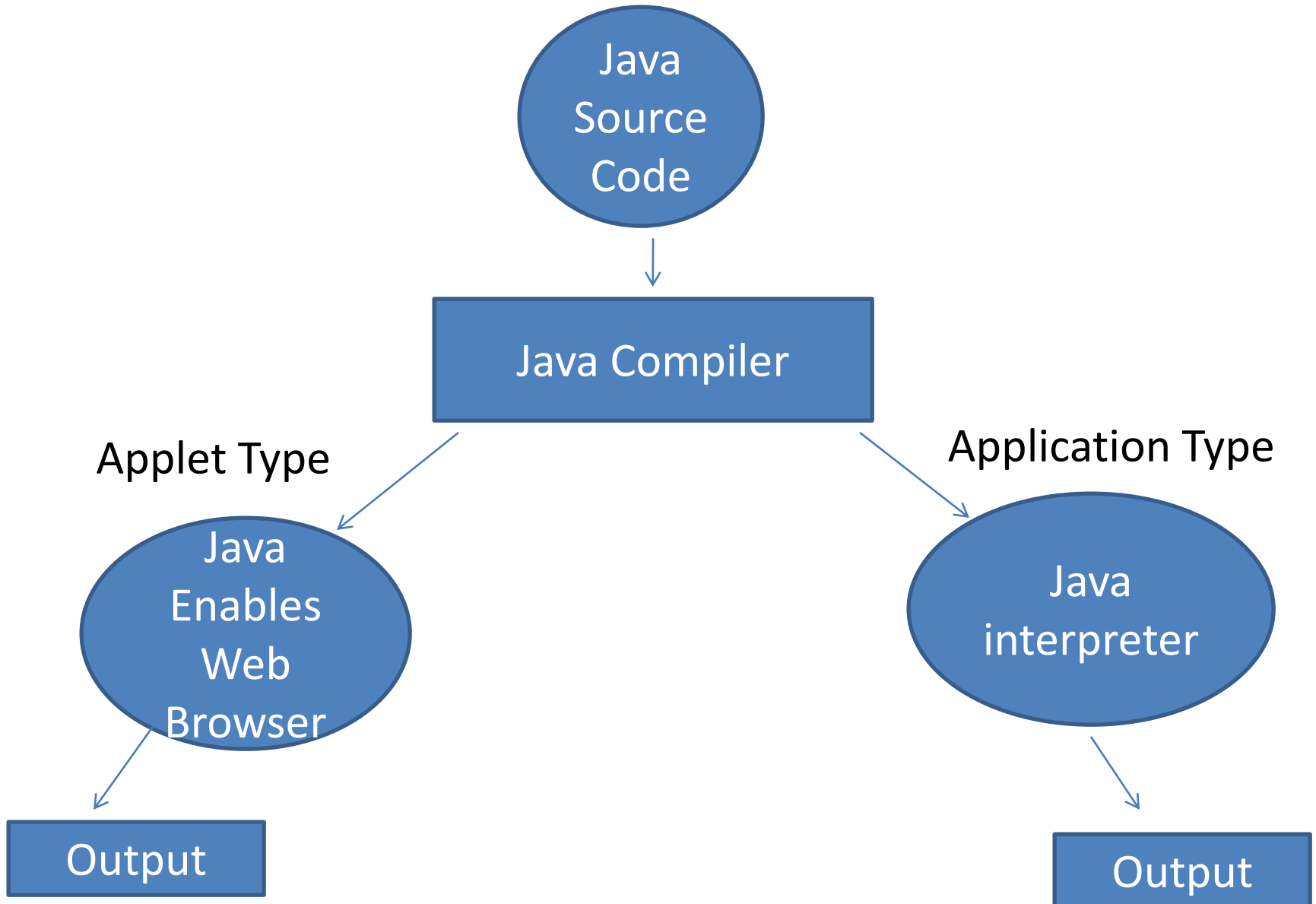
Security

- The execution of every Java program is under the control of JVM.
- It prevents the program from generating side effects outside the system.

Just In Time Compiler (JIT)

- Sun supplies Just In Time (JIT) compiler for bytecode.
- It is included in Java 2 release.
- It is part of JVM and compiles bytecode into executable code in real time, on piece by piece demand basis.
- It is not possible to compile entire Java program at once.
- Java performs various run time checks that can be done only at run time.
- JIT compiles code as it is needed during execution.

Two ways of using Java



Other Features

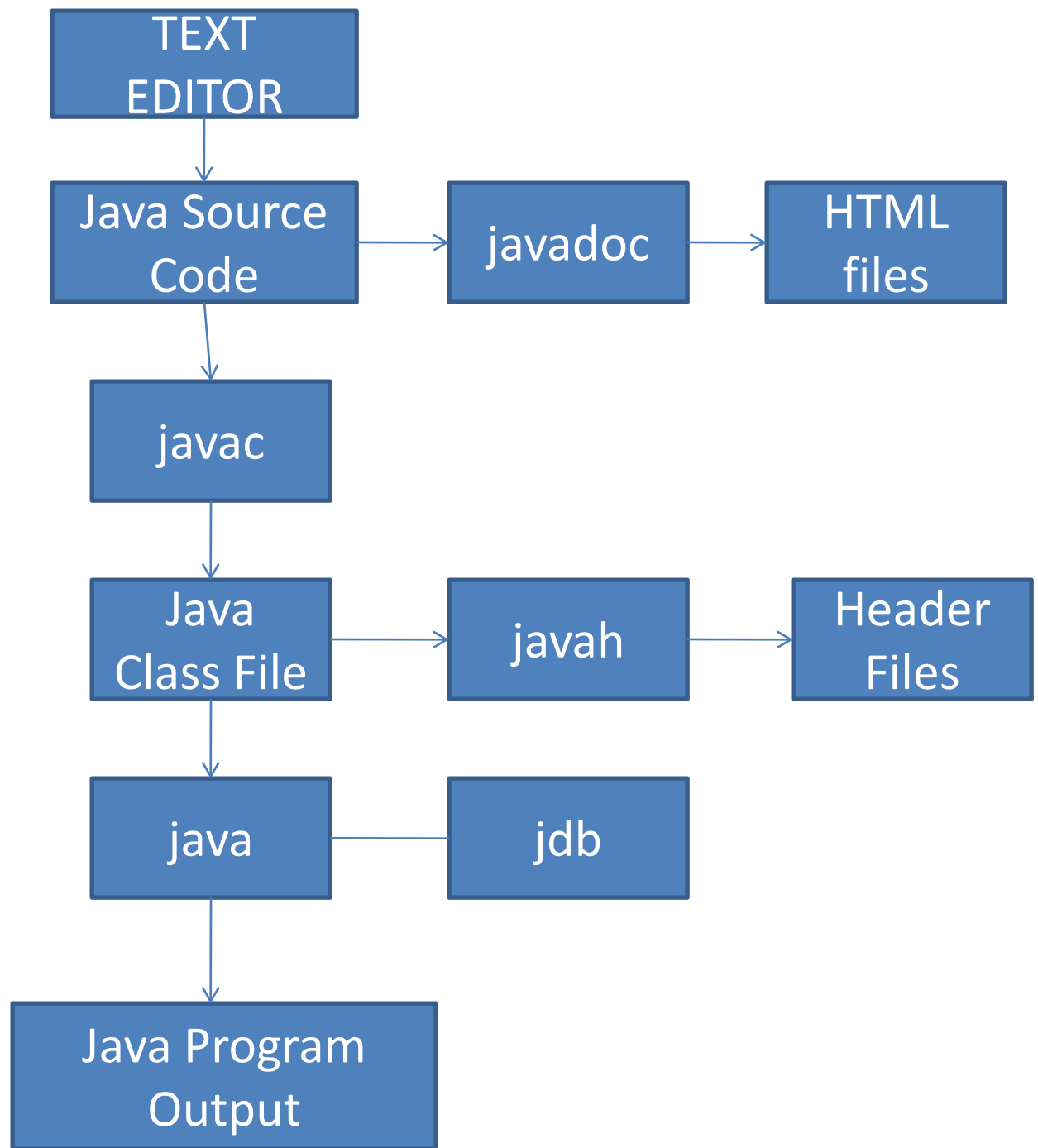
- Simple
- Secure and portable
- Object Oriented
- Robust
- Multithreaded
- Architecture neutral
- High Performance (Reduce overhead at runtime)
- Distributed (for creating applications on network)
- Dynamic

Java Environment

- It includes large number of development tools and hundreds of classes and methods.
- **Development Tools:** part of Java Development Kit (**JDK**)
- **Classes and methods:** part of Java Standard Library (**JSL**), also called Application Programming Interface (**API**)

Java Development Kit

- Comes with a collection of tools: for developing and running Java programs.
 - appletviewer (for viewing java applets)
 - javac (Java compiler)
 - java (Java interpreter)
 - javap((java disassembler): converts bytecode files into a program description.
 - javah (for c header files): produces header files
 - Javadoc (for creating html documents)
 - Jdb (java debugger): to find errors in our program



Application Programming Interface

- **Java Standard Library (or API)** includes hundreds of classes and methods grouped into several functional packages.
- Examples:
 - **Language Support Package** (for implementing basic features of java)
 - **Utilities Package** (provides utility functions such a date and time)
 - **Input/output package**
 - **Networking package**
 - **AWT package** (for implementing platform independent GUI)
 - **Applet package** (for creating java applets)

Java Runtime Environment

- JRE facilitates the execution of programs developed in Java.
- Includes:
 - **Java Virtual Machine (JVM)**
 - **Runtime class libraries:** set of core class libraries required for execution of Java program
 - **User Interface Toolkits:** that support varied input methods from users to interact with application (AWT and Swing)
 - **Deployment technologies**
 - **Java plug-in:** enables execution of applet on browser
 - **Java Web Start:** Users can launch an application directly from the web browser without going through installation procedure. (REMOTE DEPLOYMENT)

First Simple Java Program

```
public class Example
{
    public static void main (String args[])
    {
        System.out.println("Hello World !!");
        System.out.println("Simple Java Progam");
    }
}
```

Example.java

Compiling and executing

To compile this program use Java compiler

```
>>javac Example.java
```

Javac compiler creates a file called **Example.class** that contains bytecode version of the program.

To run the program use Java interpreter, java

```
>>java Example
```

Output:

Hello World !! Simple Java Program

Simple Java Program

class Example

Declares a class.

Everything must be placed inside a class.

Class is a keyword and declares that a new class definition follows.

Every class definition begins with “{” and ends with “}”.

The main method

```
public static void main (String args[])  
{  
}
```

Every Java Application program must include main() method.

This is starting point for the interpreter to begin the execution of a program.

A Java application must have any number of classes but only one class must include main() method.

The main() method

- **public:** Access Specifier. Declares it as unprotected and making is accessible to all other classes.
- **static:** this keyword declares the method as that belongs to entire class and not a part of any objects of the class.
- main method must always be declared as static since the interpreter uses this method before any objects are created.
- **void:** type modifier which specifies that the method does not return any value.

The main() method

- **The Output line**
- `System.out.println("....");`
- The `println` method is a member of the **out** object, which is a static data member of `System` class.
- **The `println`** method always appends a newline character at the end of the `String` that is passed to it.

Data Types

- Integers: this group includes **byte, short, int and long** which are for whole valued signed numbers.
- Floating point numbers: This group includes **float and double** which represent numbers with fractional precision.
- Character: This group includes **char** which represents symbols in a character set, like letters and numbers.
- Boolean: This group includes **boolean**, which is a special type for representing true/false values.

Arithmetic Operators

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Modulus (%)
- Increment (++)
- Addition Assignment (+=) (a=a+4 a+=4)
- Subtraction Assignment (-=)
- Multiplication Assignment (*=)
- Division Assignment (/=)
- Modulus Assignment (%=)
- Decrement (--)

Bitwise Operators

These are applied on integer types, long, int, short, char, byte.
These operands work on individuals bits of their operand.

- Bitwise unary NOT (~)
- Bitwise AND (&)
- Bitwise OR (|)
- Bitwise Exclusive OR (^)
- Shift Right (>>)
- Shift Right Zero Fill (>>>)
- Shift Left (<<)
- Shift Left Zero Fill (<<<)
- Bitwise AND assignment (&=)
- Bitwise OR assignment (|=)

Relational Operators

- To determine the relationship that one operand has with another
- Equal to (==)
- Not equal to (!=)
- Greater Than (>)
- Less Than (<)
- Greater than or equal to (>=)
- Less than or equal to (<=)

Boolean Logical Operators

- Operate on boolean operands.
- All the binary logical operators combine two boolean values to form a resultant boolean value.
- Logical AND (&)
- Logical OR (|)
- Logical XOR (^)
- Short Circuit OR (||)
- Short circuit AND (&&)
- Logical unary NOT (!)
- AND Assignment (&=)
- OR Assignment (|+)
- XOR Assignment (^=)
- Equal To (==)
- Not Equal To (!=)
- Ternary if-then-else (? :)

Assignment & conditional operator

Assignment operator (=)

var = expression;

Conditional Operator (ternary operator)

expression 1 ? expression 2 : expression 3

float ratio;

int num=25; denom=5;

ratio = denom==0 ? 0 : num/denom;